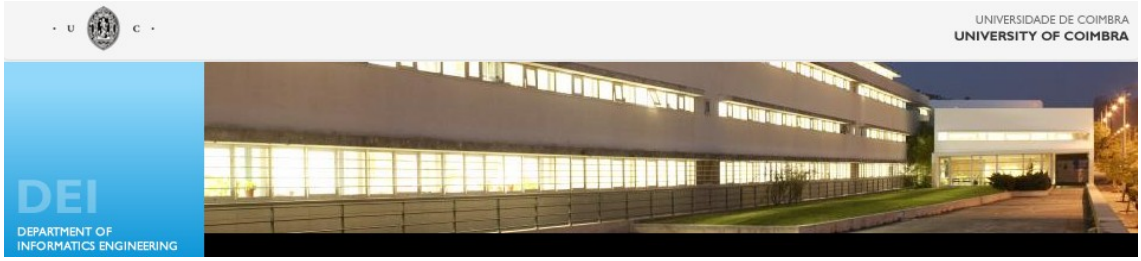

Desempenho de Serviços e Redes de Comunicação

(Communication Networks and Services Performance) [2025-2026]

Exercise 02 – Getting performance data from the network

Introduction

This exercise aims at learning how to manipulate and plot data using Pandas and to introduce variance to simulations.

Objectives

Students successfully concluding this work should be able to:

- Manipulate raw simulation data from YAFS to analyse results
- Generate traffic dynamically
- Automate simulation execution

Submission

Groups of 2 students shall submit the work in *Inforestudante* as a .zip, containing the following files:

- 1 page A4 report detailing the process used on each exercise
- All the plots obtained
- All the Python files changed/created

Submission deadline: **April 1st, 2026**.

Background

Pandas

Pandas[1] is a fast, powerful, flexible and easy to use open-source data analysis and manipulation tool, built on top of the Python programming language.

Pandas provides two types of classes for handling data:

- Series: a one-dimensional labeled array holding data of any type such as integers, strings, Python objects etc.
- DataFrame: a two-dimensional data structure that holds data like a two-dimension array or a table with rows and columns.

Customarily, we import as depicted in Figure 1.

```
import numpy as np

import pandas as pd
```

Figure 1. Pandas import

Numpy

NumPy[2] is a Python library used for working with arrays. It also has functions for working in domain of linear algebra, fourier transform, and matrices. Numpy is an open-source project, and you can use it freely. NumPy stands for Numerical Python.

Confidence intervals

A Confidence Interval (CI) is an interval which is expected to typically contain the parameter being estimated. More specifically, given a confidence level γ (95% and 99% are typical values), a CI is a random interval which contains the parameter being estimated γ% of the time

[1] https://pandas.pydata.org/
[2] https://numpy.org/

Familiarization with Pandas

1. Installing and importing Pandas

Install it using this command:

```
$>pip install pandas
```

If this command fails, then use a python distribution that already has Pandas installed like, Anaconda, Spyder etc.

Once Pandas is installed, import it in your applications by adding the import keyword:

```
import pandas as pd
```

2. Data processing using pandas

Now you should be able to use Pandas. Try the following example:

```
import pandas as pd

mydataset = {
             'students': ["Joao", "Luis", "Ana"],
             'grades': [14, 09, 19]
            }

myvar = pd.DataFrame(mydataset)

print(myvar) #What do you see?
```

You can also use Series, a one-dimensional array (similar to a column in a table).

```
import pandas as pd

a = [1, 7, 2]

myvar = pd.Series(a)

print(myvar) #What do you see?
print(myvar[0]) #What do you see?
```

You can create labels to access a particular item.

```
import pandas as pd

a = [1, 7, 2]

myvar = pd.Series(a, index = ["x", "y", "z"])

print(myvar)
```

TRY IT YOURSELF: Create a Pandas Series from a dictionary.

Data sets in Pandas are usually multi-dimensional tables, called DataFrames. Series is like a column, a DataFrame is the entire table.

TRY IT YOURSELF: Create a Pandas DataFrame from two Series.

Important attributes and arguments

Pandas use the loc attribute to return one or more specified row(s). With the index argument, you can name your own indexes.

3. Importing data to pandas from a file

If your data sets are stored in a file, Pandas can load them into a DataFrame. See the following example:

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df)
```

TRY IT YOURSELF: Use the file data.csv available with this exercise and try to load it into a DataFrame.

TRY IT YOURSELF: Use the file data.json available with this exercise and try to load it into a DataFrame.

If your JSON code is not in a file, but in a Python Dictionary, you can load it into a DataFrame directly.

TRY IT YOURSELF: Create a Python dictionary and load it into a DataFrame.

4. Analysing DataFrames using pandas

One of the most used method for getting a quick overview of the DataFrame, is the head() method. The head() method returns the headers and a specified number of rows, starting from the top. There is also a tail() method for viewing the last rows of the DataFrame. The tail() method returns the headers and a specified number of rows, starting from the bottom.

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.head(10))
print(df.tail())
```

TRY IT YOURSELF: Print the first 5 rows and the last 5 rows of the DataFrame in the data.csv file. Try the info() method for the DataFrame. What do you see?

5. Plotting data with Pandas

Pandas uses the plot() method to create diagrams. We can use Pyplot, a submodule of the Matplotlib library to visualize the diagram on the screen. Try the following example (using the data.csv file):

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('data.csv')

df.plot()

plt.show()
```

TRY IT YOURSELF: See the documentation[3] and try different plot types (parameter kind).

[3] https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.plot.html

Activities

1. Plotting YAFS results with Pandas

Goal: Learn how to integrate YAFS with Pandas.

YAFS simulations generate two raw data results files in .csv format. The first file is called `sim_trace.csv` and it has a register of each message in each entity of the topology. The second file called `sim_trace_link.csv` holds the transmission process in the network.

Analyze both files generated by `ex1.py` and see what they have.

Submit a plot reporting the latency of the applications deployed in the network and the created Python script to analyze the data.

What are your conclusions?

2. Dynamize your simulations

Goal: Learn how to create simulations with dynamic traffic.

YAFS allows for deterministic (or static) and variable (or dynamic) generation of traffic. While deterministic generation of traffic is useful for benchmarks and validation of scenarios, it is not realistic to be used in realistic simulations.

The simulator provides two generator classes:

- `exponential_distribution` (for Poisson processes)
- `uniformDistribution` (for uniform distributions)

Change the simulation in ex2_3.py to introduce dynamic traffic generation for each class. **Do not change the existing seed.**

Submit a plot reporting the latency of the applications for each generator and the original generator, and the created Python script to analyze the data.

What are your conclusions? Describe the values used for each generator class.

3. Automating your simulations

Goal: Learn how to automate YAFS simulations.

When using simulations, one must be aware of statistical errors. The smaller the sample size, the greater the average effect size of false positive results (i.e., those that result in significant results due to chance and random situations). To avoid statistical error, we must run several simulations, reporting the average value of the results from the different simulations.

Use the provided Python file used in the previous exercise to run the simulation a variable number of times. Use Poisson and uniform distributions with the same parameters as the previous exercise.

As each simulation produces a distinct file set, the plot should present the averaged values across all datasets. This also allows you to report confidence intervals. Consider **10**, **30, 100 and 1000** runs. **Do not forget to change the seed to a different number on each run.**

Submit the plots reporting the average total latency of the applications deployed in the network for all sets of runs, and the created Python script to analyze the data. Include 95% confidence intervals.

What are your conclusions?